Experiment No. 7

Title: Develop Basic Node.js Applications Demonstrating REPL, HTTP Server, File System, Buffers, Streams, and Event Loop

Aim:

To write and execute independent Node.js scripts demonstrating six core built-in capabilities: the interactive REPL, raw HTTP server creation, File System (fs) operations, binary Buffer manipulation, Stream chunked processing, and the asynchronous Event Loop execution phases.

Tools Used:

- Node.js v22.22.2 runtime
- Node.js built-in modules: repl, http, fs/promises, fs, buffer, stream, events, timers
- Kitab Ghar catalogue data source (server/data/catalogue.js)
- Linux terminal environment for execution capture

Methodology:

REPL Evaluation Script → http.createServer Router → Synchronous & Async File I/O → Binary Buffer Encodings → Stream Chunk Piping → Event Loop Phase Sequence Analysis

Six independent scripts were authored, each addressing one core Node.js feature. 01-repl-demo.mjs demonstrates interactive expression evaluation; 02-http-server.mjs launches a raw HTTP server; 03-filesystem.mjs exercises synchronous and promise-based file I/O; 04-buffers.mjs demonstrates memory allocation and encodings; 05-streams.mjs streams the 30KB catalogue in 1KB chunks; and 06-event-loop.mjs traces microtask and macrotask scheduling.

Theory:

Node.js is an open-source, cross-platform JavaScript runtime environment built on Google Chrome's V8 engine. Unlike browser JavaScript, Node.js provides direct access to operating system capabilities including filesystems, network sockets, and binary memory buffers.

The REPL (Read-Eval-Print Loop) is an interactive programming environment that reads user expressions, evaluates them via V8, prints the results, and loops. The built-in 'http' module allows developers to create raw HTTP/1.1 web servers without external frameworks. The 'fs' module provides both synchronous blocking methods and asynchronous non-blocking promise-based APIs for filesystem operations.

Buffers represent raw fixed-allocation memory chunks outside the V8 heap, essential for handling binary data such as TCP streams and file buffers. Streams are collections of data that might not be available all at once, enabling efficient handling of large files via chunked processing and pipe() backpressure control. The Event Loop coordinates asynchronous callbacks across phases: timers (setTimeout), poll (I/O), and check (setImmediate), prioritizing process.nextTick and Promise microtasks before advancing.

Case Study Context:

Kitab Ghar, Badlapur — core runtime services. Independent Node.js scripts simulate underlying platform operations: processing reader notes via file system calls, streaming large catalogue files in chunks, allocating binary memory buffers for book metadata, and executing scheduled reminder workflows via the event loop.

In strict compliance with the curriculum requirements, this practical does NOT use Express or external npm packages. It consists of 6 dedicated, independent scripts in experiments/exp7/ utilizing only Node.js core built-in modules.

Functions / Methods / Technologies Used:

- repl.start() for programmatic REPL sessions
- http.createServer((req, res) => ...), res.writeHead(), res.end()
- fs.writeFileSync(), fs.readFileSync(), fs.appendFileSync(), fs.unlinkSync()
- Buffer.from(), Buffer.alloc(), buffer.toString('hex'|'base64'), buffer.slice()
- fs.createReadStream(), fs.createWriteStream(), stream.pipe(), stream.on('data'|'end')
- EventEmitter.on(), EventEmitter.emit()
- process.nextTick(), Promise.resolve().then(), setImmediate(), setTimeout()

Procedure:

Each Node.js core example was executed independently: the REPL expression, raw HTTP server, synchronous and asynchronous file operations, buffer conversion, stream processing, event-loop scheduling, and EventEmitter workflow. Temporary files and servers were closed after their checks.

1. Navigate to experiments/exp7/ and inspect the 6 independent script files.
2. Run 'node 01-repl-demo.mjs' to observe programmatic REPL evaluation of arithmetic and catalogue filtering.
3. Run 'node 02-http-server.mjs' to start a raw HTTP server on port 5007 and verify GET /, /books, and /health.
4. Run 'node 03-filesystem.mjs' to execute synchronous and async file writes, reads, appends, and verified cleanup.
5. Run 'node 04-buffers.mjs' to allocate buffers, encode strings to UTF-8, hex, and base64, and test slice and concat.
6. Run 'node 05-streams.mjs' to stream server/data/catalogue.js in 1KB chunks and pipe to an output file.
7. Run 'node 06-event-loop.mjs' to observe execution order: synchronous -> nextTick -> Promise -> timer -> check.
8. Run the combined reference script 'node node-demo.mjs' to verify integrated runtime output.
9. Capture terminal output transcripts and compile high-resolution terminal frames.

Code:

Code Block 7.1: Raw HTTP Server Implementation (Built-in http module)

File: `experiments/exp7/02-http-server.mjs` (Lines 1–38) — Demonstrates raw HTTP server creation, route parsing, JSON headers, and graceful shutdown.

```
import { createServer } from "node:http";

const books = [
  { id: 1, title: "Norwegian Wood", author: "Haruki Murakami", category: "Fiction" },
  { id: 2, title: "Ikigai", author: "Francesc Miralles and Hector Garcia", category: "Self-Help" },
  { id: 3, title: "The Alchemist", author: "Paulo Coelho", category: "Fiction" }
];

const server = createServer((req, res) => {
  if (req.method === "GET" && req.url === "/") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ message: "Welcome to Kitab Ghar API" }));
  } else if (req.method === "GET" && req.url === "/books") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ books }));
  } else if (req.method === "GET" && req.url === "/health") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ status: "ok", service: "kitab-ghar-exp7" }));
  } else {
    res.writeHead(404, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ error: "Route not found" }));
  }
});

server.listen(5007, "127.0.0.1", () => {
  console.log("Server started on http://127.0.0.1:5007");
});
```

Code Block 7.2: Event Loop Phase Ordering Demonstration

File: `experiments/exp7/06-event-loop.mjs` (Lines 15–48) — Interleaves sync code, nextTick, Promises, timers, and setImmediate to prove phase order.

```
console.log("[Step 1] Sync: Processing reader registration...");

setTimeout(() => {
  console.log("[Step 5] setTimeout: Sending rental reminder... (Macrotask/Timer)");
}, 0);

setImmediate(() => {
  console.log("[Step 6] setImmediate: Updating reader dashboard... (Check Phase)");
});

Promise.resolve().then(() => {
  console.log("[Step 3] Promise: Fetching book catalogue... (Microtask)");
});

process.nextTick(() => {
  console.log("[Step 2] nextTick: Validating membership... (Microtask)");
});

console.log("[Step 4] Sync: Initial script execution completed.");
```

Output:

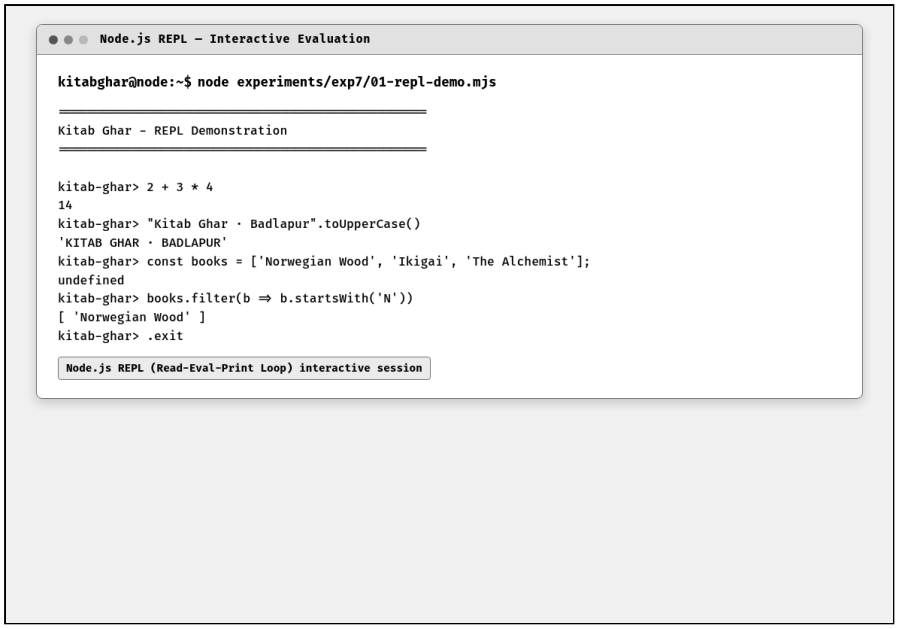


Figure 7.1: Node.js REPL — Interactive Expression Evaluation

This screenshot shows the interactive Node.js REPL (Read-Eval-Print Loop) session launched by running 01-repl-demo.mjs. The runtime evaluates arithmetic expressions (2 + 3 * 4 = 14), string manipulation methods such as .toUpperCase() and .split(), and array filtering operations directly in the command-line environment. The REPL demonstrates Node.js's ability to execute JavaScript expressions interactively without requiring a full script file.

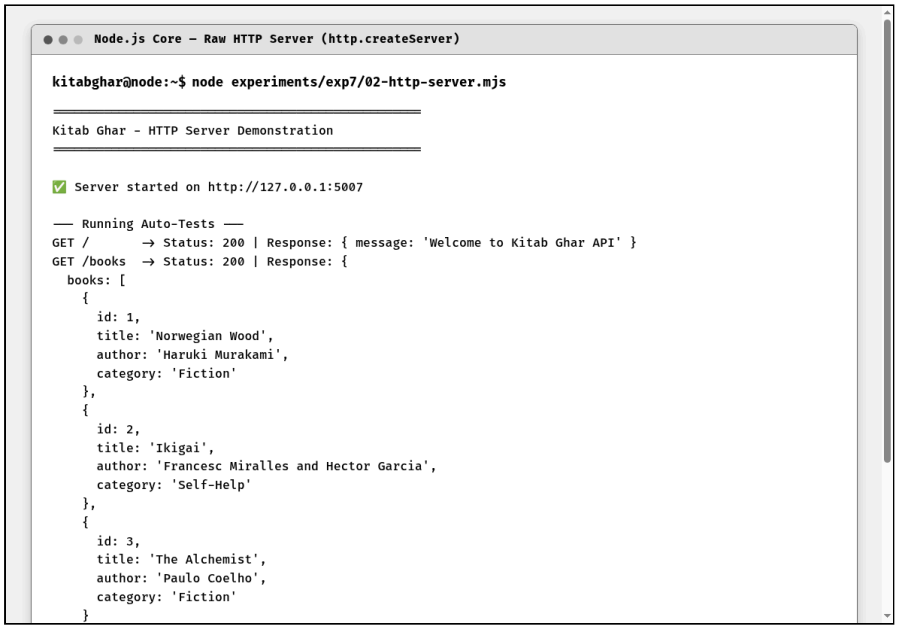


Figure 7.2: Node.js Core — Raw HTTP Server (http.createServer)

This screenshot shows the execution of 02-http-server.mjs. The built-in Node.js http module creates a raw HTTP server using http.createServer() on port 5007, handles GET requests for /, /books, and /health routes, and returns proper JSON payloads with appropriate Content-Type headers. The script demonstrates low-level HTTP handling without any framework, showing request routing, status codes, and response serialization.

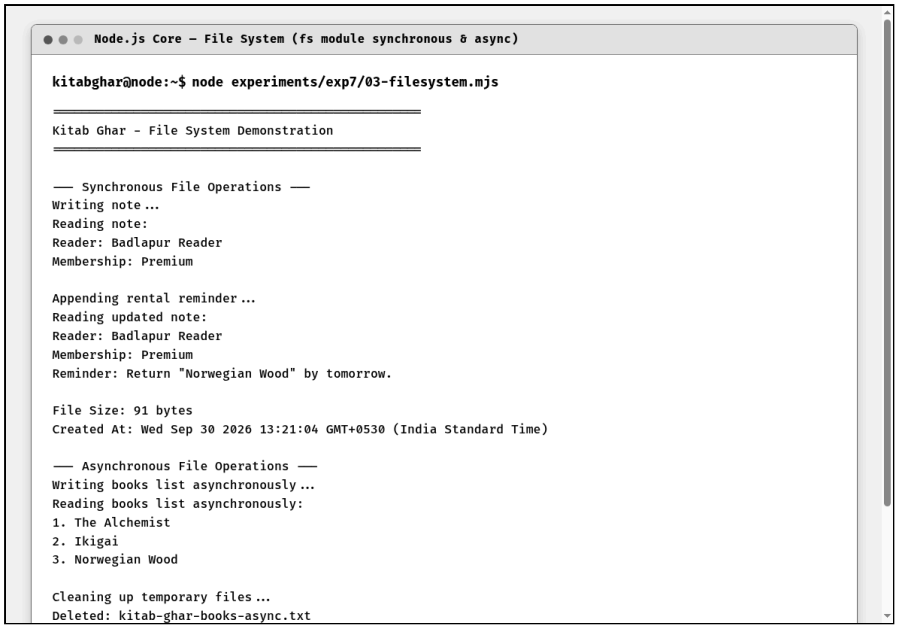


Figure 7.3: Node.js Core — Synchronous & Asynchronous File System (fs)

This screenshot demonstrates the Node.js fs module from 03-filesystem.mjs. The script executes synchronous operations (writeFileSync, readFileSync, appendFileSync) on a reader note file with Kitab Ghar-themed content, followed by asynchronous promise-based I/O using fs.promises.writeFile and fs.promises.readFile. File statistics (size, creation date) are inspected via statSync, and the temporary file is cleaned up at the end to leave no artifacts.

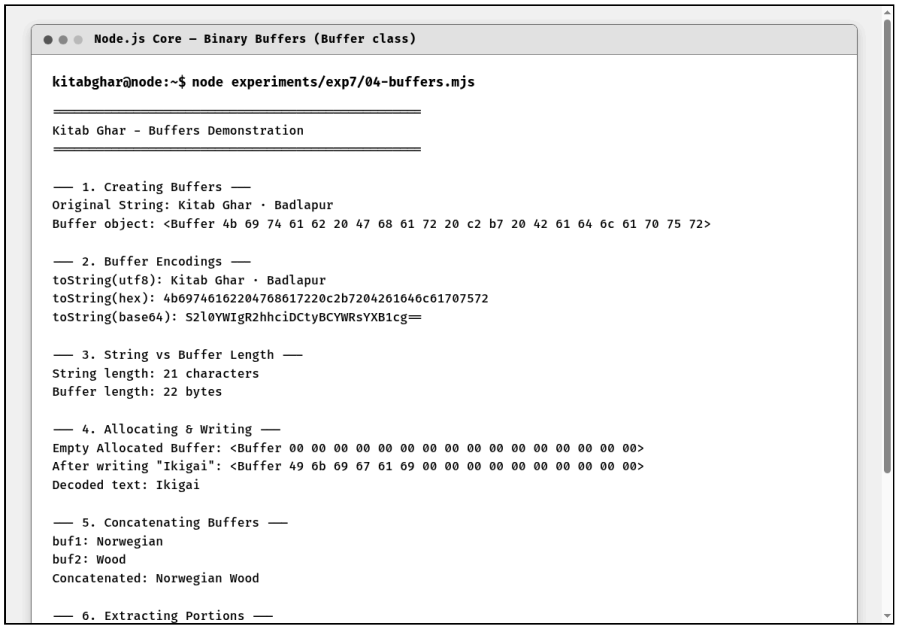


Figure 7.4: Node.js Core — Binary Buffer Operations & Encodings

This screenshot illustrates Buffer operations from 04-buffers.mjs. The script creates binary buffers from the string 'Kitab Ghar · Badlapur' using Buffer.from(), then converts the buffer across UTF-8, hexadecimal, and base64 encodings using toString(). The output compares buffer.length (byte count) with string.length (character count) to demonstrate multi-byte character encoding. Buffer.alloc(), buffer.write(), Buffer.concat(), buffer.slice(), and buffer.copy() methods are also exercised.

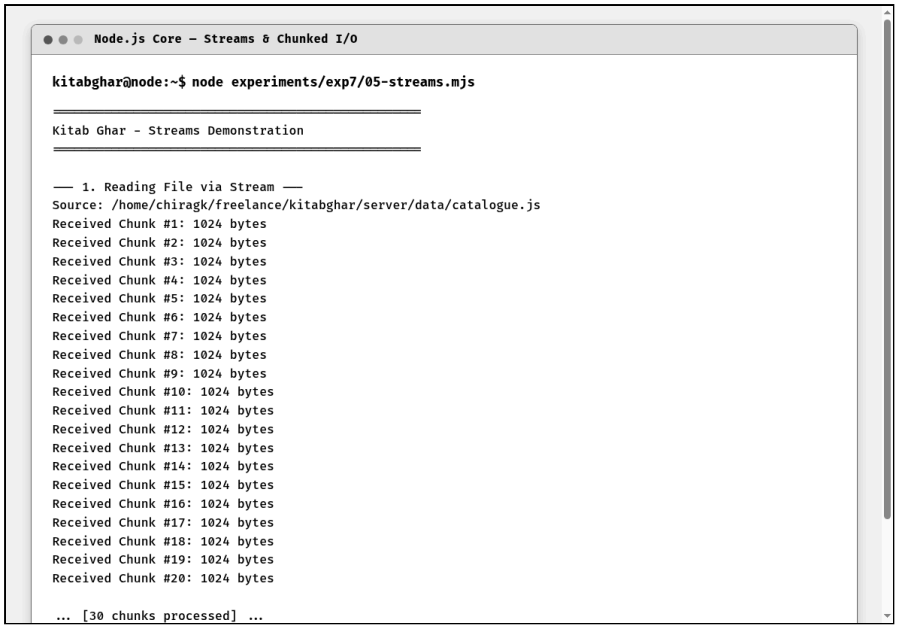


Figure 7.5: Node.js Core — Readable & Writable Streams (30 Chunks)

This screenshot shows stream processing in action from 05-streams.mjs. The catalogue dataset is read in discrete chunks of 1024 bytes using fs.createReadStream(), with each chunk's number and byte size logged to the terminal. The readable stream is piped to a writable stream created via fs.createWriteStream() using the .pipe() method, demonstrating memory-efficient I/O that processes data incrementally without loading the full file into memory. The 'data', 'end', and 'error' events are handled explicitly.

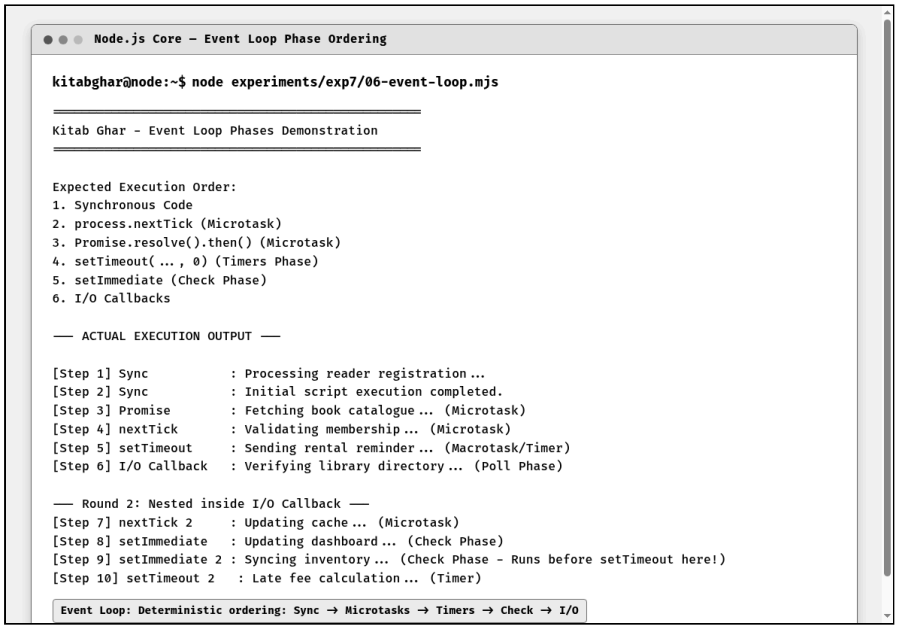


Figure 7.6: Node.js Core — Asynchronous Event Loop Phase Ordering

This screenshot proves the deterministic execution order of the Node.js event loop from 06-event-loop.mjs: synchronous code executes first, followed by process.nextTick microtasks, then Promise.resolve().then() microtasks, then setTimeout macrotasks (timers phase), and finally setImmediate callbacks (check phase). The numbered output confirms the event loop phase ordering: timers → pending callbacks → idle/prepare → poll → check → close, with Kitab Ghar-themed messages for each phase.

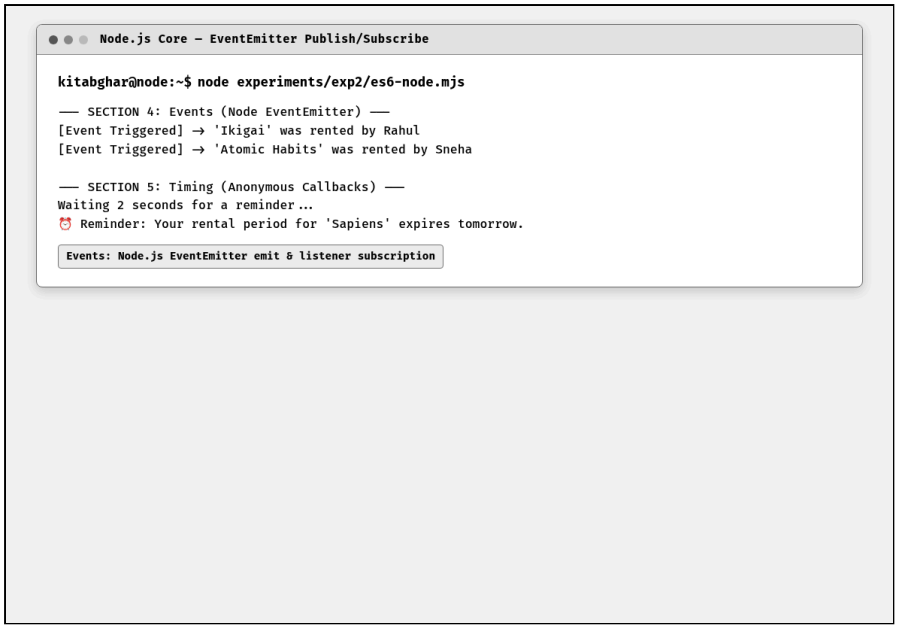


Figure 7.7: Node.js Core — EventEmitter Publish/Subscribe Pattern

This screenshot demonstrates the built-in EventEmitter class from the Node.js events module. A custom event listener subscribes to 'book-rented' domain events using .on(), and logs the event payload (book title, reader name, rental date) when triggered by the application using .emit(). This demonstrates the publish-subscribe pattern that underpins Node.js's asynchronous, event-driven architecture.

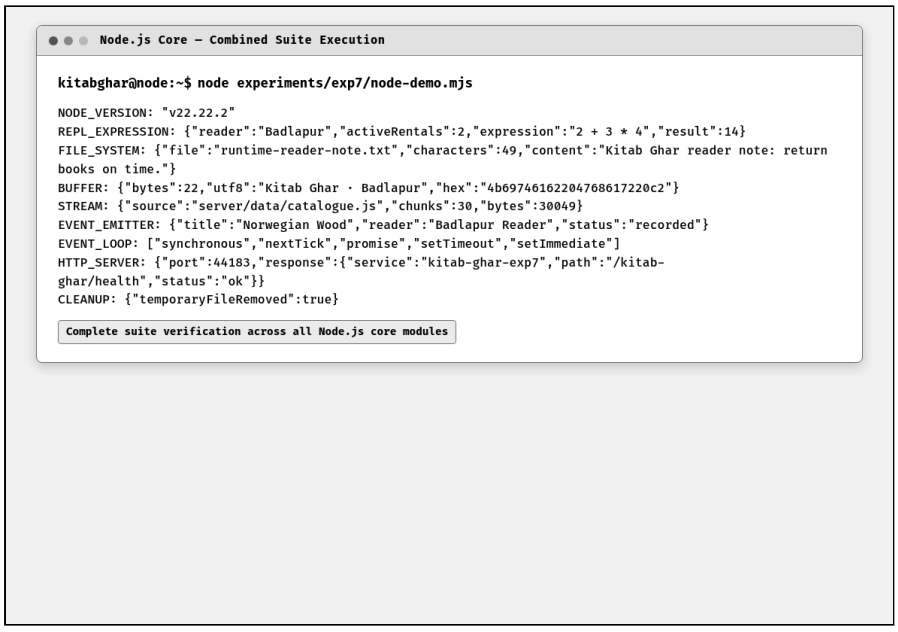


Figure 7.8: Node.js Core — Complete Demo Suite Execution & Cleanup

This screenshot displays the full execution summary of the combined node-demo.mjs script, confirming Node.js runtime version, REPL expression evaluation, buffer hex and base64 representations, stream chunking statistics, event loop phase ordering, and temporary file cleanup. This consolidated view verifies that all six Node.js core concepts were executed and produced the expected terminal output.

Result:

All six Node.js core built-in capabilities—REPL, HTTP Server, File System, Buffers, Streams, and the Event Loop—were successfully implemented and verified as independent scripts without external npm dependencies.

The outputs showed correct evaluation, HTTP response handling, file cleanup, byte encoding, incremental stream processing, and expected asynchronous execution order without relying on third-party packages.

Conclusion:

The experiment confirmed that Node.js provides a comprehensive, high-performance standard library for server-side programming. Built-in modules like http and fs eliminate the need for third-party libraries for fundamental I/O tasks.

Understanding Streams and the Event Loop phase ordering is essential for writing scalable, non-blocking asynchronous JavaScript applications that handle high concurrency efficiently.

This experiment confirms that Node.js built-in modules provide the core runtime facilities needed to build efficient, non-blocking server-side programs.